Table of Contents

Guide utilisation 2

Dataset configurator 2

Charger un dataset 2

Visualiser les données 3

Traiter les données 4

Cas d'utilisation typique 7

Outil d'entrainement de modèle 8

Chargement du modèle pré-entrainé 8

Chargement du dataset 9

Choix des Hyperparamètres 9

Augmentation de données 10

Résultats 11

Utiliser le modèle 12

Initialisation 12

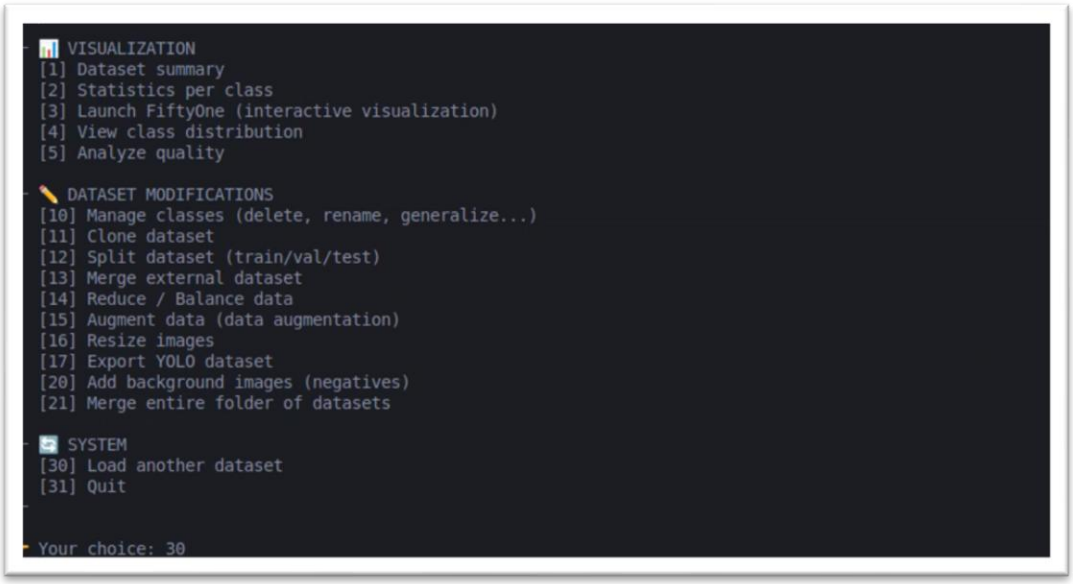
Choix du mode de reconnaissance 12

Guide utilisation

Dataset configurator

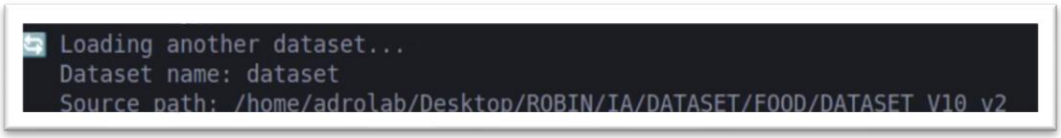
Le programme dataset configurator permet d'affiner un modèle au format YOLO pour pouvoir l'entrainer.

Interface générale en anglais



Charger un dataset

Rien de plus simple, le programme demande le chemin du fichier de configuration yaml. Ce fichier répertorie toutes les classes du dataset :

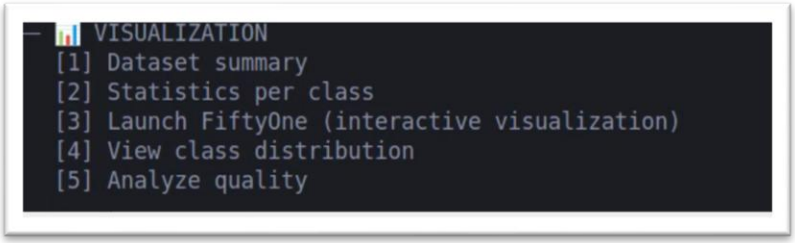


Le programme permet de fusionner le dataset chargé avec un autre existant.



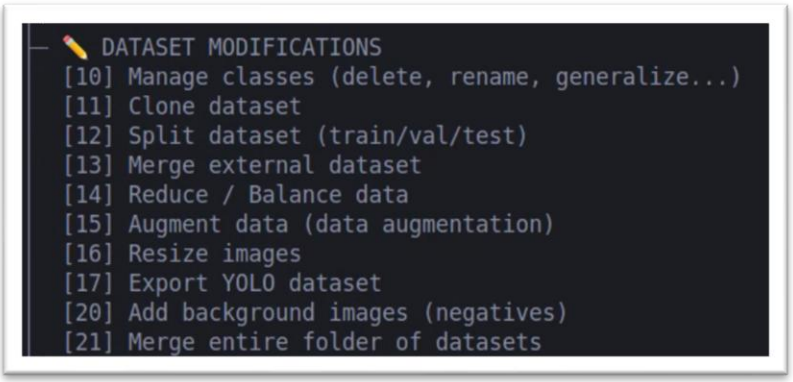
Visualiser les données

Plusieurs options pour analyser les données sont mises à disposition :



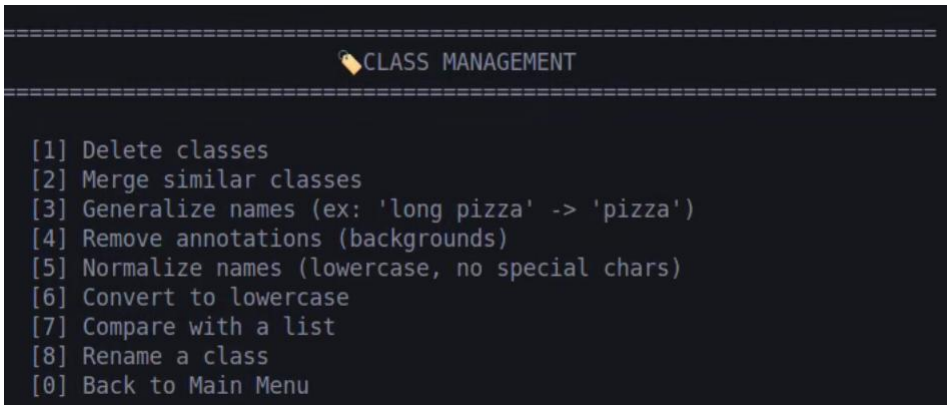
- [1] un simple résumé du dataset (nombre d'images & classes)
- [2] Nombre d'images par classe dans un fichier texte
- [3] Ouvre l'outil FiftyOne (éditeur de dataset)
- [4] Affiche un tableau avec le nombre d'images par classe
- [5] Donne les écarts types entre les classes, les classes non labélisées…

Traiter les données



Class Management [10]

Options pour éditer les classes



[1] Supprimer une ou plusieurs classes

[2] Fusionner des classes entre elles (dans le cas où elles seraient similaires, ex : apple & green apple)

[3] Les noms sont généralisées

[4] Enlever les labels d'une classe -> la classe devient une classe « témoin » c-à-d qu'elles ne font plus parties des classes que le modèle doit reconnaitre.

Cela permet au modèle de s'entrainer sur des images où il n'y a pas forcément des objets à reconnaitre

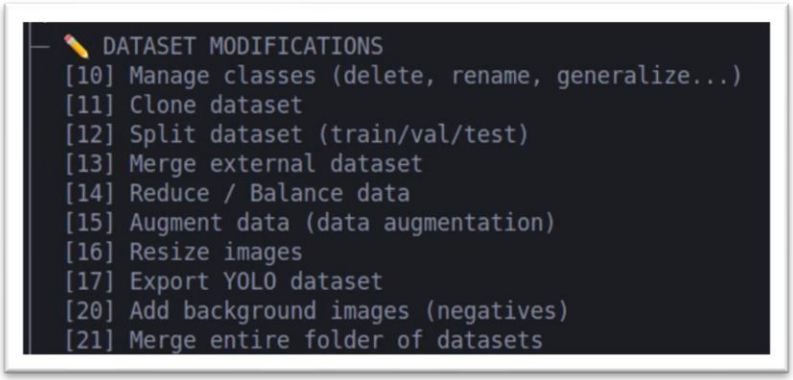
[5] Formater le nom des classes (suppression des virgules, mises en minuscules …)

[6] Modifie le nom des classes en les mettant **seulement** en miniscule

[7] L'utilisateur rentre une liste de classes et la fonction renvoie les classes en commun et celles absentes

[8] Permet de renommer une classe spécifique

Autres fonctions de traitement



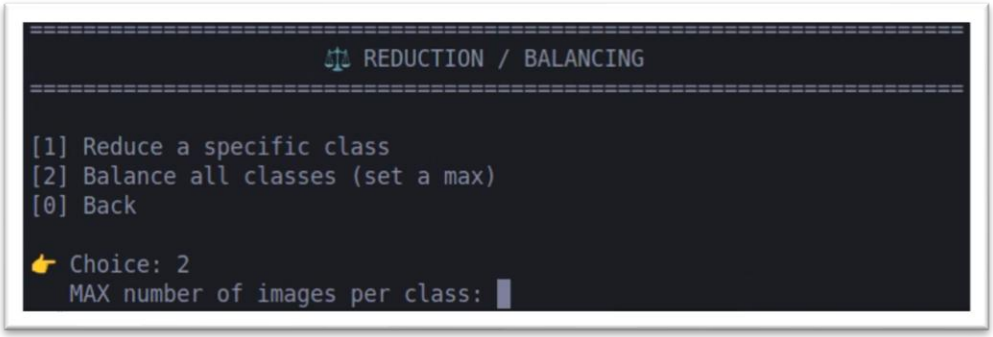
[11] Permet de cloner le dataset (et ne pas modifier l'instance actuelle)

[12] Permet de modifier le ratio de tri entre les images sevant à l'entrainement (train), à la validation (valid) et au test final (test).

Par défaut sur 0.7, 0.2, 0.1

[13] Permet de fusionner avec un autre dataset

[14] Permet de fixer un nombe n images par classe (et ainsi éviter les déséquilibres dans le dataset)



[15] Outils de data augmentation, permet à partir d'échantillons d'images existants d'en créer de nouveaux en effectuant du traitement de l'image (luminosité, saturation, rotation, translation) ...

-> Permet d'augmenter artificiellement le nombre d'images d'une classe qui serait en sous-nombre d'images

-> Permet de créer des échantillons uniques des fois pour forcer l'adaptation du modèle

[16] Permet de modifier la résolution des images (par défaut en 640x640), certains usages demandent une plus haute résolution pour de bons résultats.

**/! \ Les performances sont également plus exigeantes /! **

[17] Permet l'export du dataset dans un répertoire donné

[20] Permet de sélectionner un dossier d'images afin de les ajouter au dataset en tant qu'images « background »

-> images qui vont servir d'images « témoins » afin d'éviter que le modèle voie des objets là où il n'y en a pas.

[21] Permet de fusionner **tous** les datasets se trouvant dans un dossier spécifique.

Cas d'utilisation typique

L'utilisation typique de l'outil pour modifier un dataset se fait ainsi :

1) Import du dataset

Optionnel : Fusion avec d'autres datasets

2) Modification des classes (suppression des classes inutiles, formatage des noms …)
3) Modification des images : Fixer un nombre n images par classe pour éviter le déséquilibre du dataset
4) Export du dataset

Les fonctions comme le **changement du split du dataset** (par défaut en 0.7, 0.2, 0,1) , le changement de résolution des images ou l'ajout d'images backgounds sont utiles dans des cas bien spécifiques.

Enfin les fonctions de **data augmentation** sont également intégrées lors du processus d'entrainement et il souvent préférable de les employer à ce moment-là hors cas spécifique.

(Cela évite de créer de nouveaux échantillons à partir d'échantillons qui étaient déjà des copies)

Outil d'entrainement de modèle

Chargement du modèle pré-entrainé

Une fois le dataset configuré on peut passer à l'entrainement.

L'entrainement se fait par le fichier **train.py**

Il faut pour cela charger le fichier contenant les poids du modèle voulu,

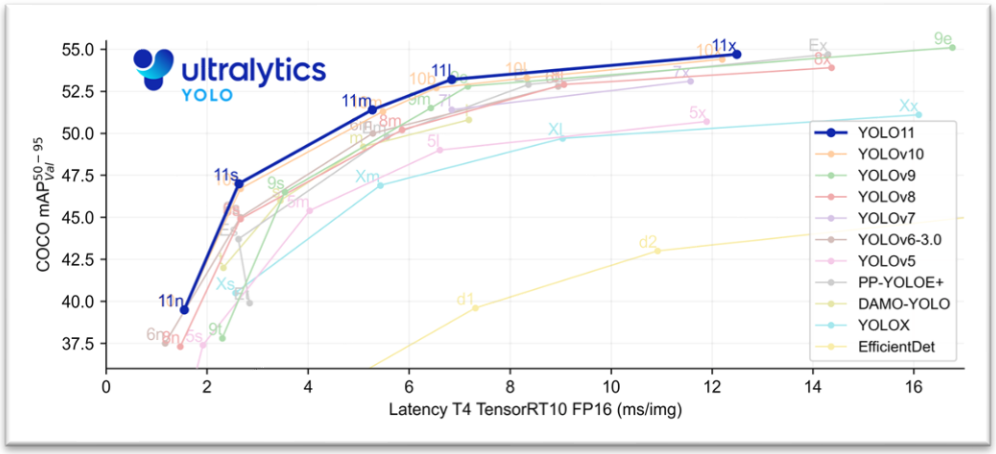
YOLO a publié plusieurs modèles types, voici comment les différencier.

yolo11x.pt
11 : numéro de version
x : type de modèle

n → nano
s → small
m → medium
l → large
x → extra large

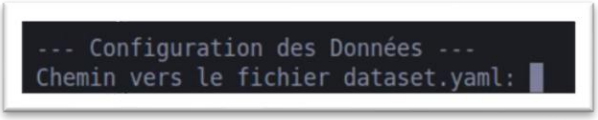
Chacun a un usage type, à savoir que plus le modèle est performant plus il est lourd en termes de ressources.

Dans le contexte de reconnaissance d'aliments, les détails étant très importants, une version **medium – large est donc recommandée.**



Chargement du dataset

Le chargement du dataset se fait simplement par l'envoie du répertoire où se trouver la fonction de configuration YAML.



Choix des Hyperparamètres

Epochs

L'entraînement du modèle repose sur la répétition de « n » époques.

Chaque époque représente un cycle complet durant lequel l'algorithme cherche à optimiser ses **poids** sur le jeu d'entraînement (split train), avant de les tester sur le jeu de validation (split valid).

Valeurs recommandées : 20 – 150

- 20 pour un petit modèle avec peu de données
- 150 pour un modèle avec beaucoup de données

**/! \ un nombre trop important peut entrainer un sur-apprentissage / ! **

Resolution

Permet de choisir la résolution, à adapter en fonction des images se trouvant dans le dataset

Batchsize

Nombre d'images traités en même temps par le modèle avant qu'il mette à jour ses poids :

Avec un gros batch size, cela permet au modèle de bien différencier les classes entre elles.

Cependant, le batch size est élevé plus la mémoire RAM de la carte graphique est utilisée.

Augmentation de données

Permet la configuration des augmentations de données à savoir :

Modifications graphiques :

• la teinte, la saturation, la luminosité

Modifications géométriques :

• rotations, translations, échelle

Modifications supplémentaires :

Mosaic : permet de fusionner 4 images en une pour que le modèle reconnaisse les différents objets présents.

Mixup : permet de superposer deux images entre elles

Patience

Le paramètre de patience permet d'arrêter l'entrainement si l'indice map50 n'augmente pas pendant n epochs.

Résultats

Les résultats sont publiés au fur et a mesure dans le répertoire donné par l'utilisateur c-a-d le **réseau de neurones** sous deux versions :

Last.pt : derniers poids obtenus à la fin de la dernière epoch

Best.pt : poids avec le score map50 le plus élevé.

Un fichier csv permet de suivre la performance du modèle : results.csv



Utiliser le modèle

Initialisation

Une fois le modèle créé, on peut maintenant l'utiliser dans le programme main.py.

Le programme main.py est accompagné de **configure.py & config.py**

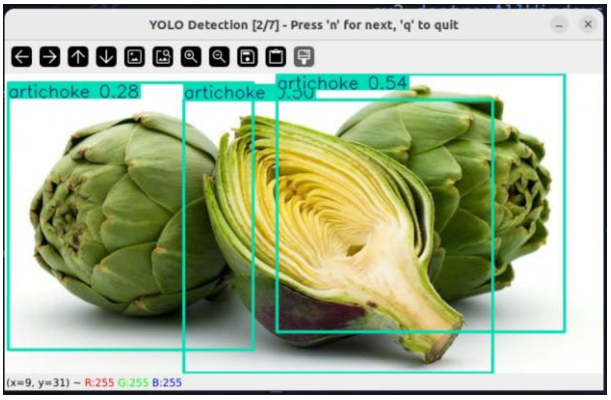
• Config.py contient les paramètres pour l'exécution du programme main (répertoire du modèle, de la caméra)

• Configure.py permet de modifier intuitivement le fichier config.py

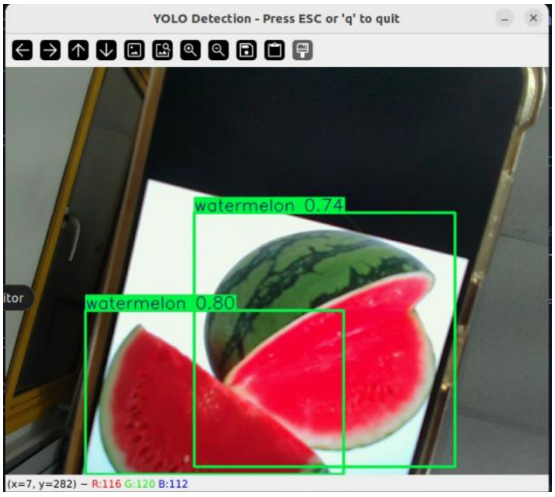
Choix du mode de reconnaissance

Trois choix s'offrent pour la reconnaissance, cela peut être de la reconnaissance :

• d'une simple image ou d'un dossier contenant plusieurs images

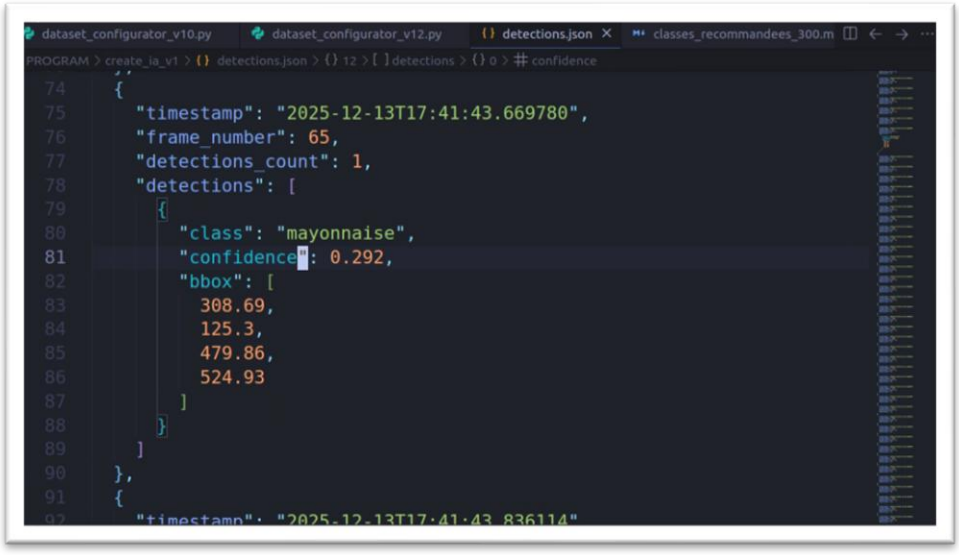


• d’un stream en temps via la caméra



/!\Attention au choix de la caméra (Inter RealSense) ou classique et à bien renseigner le port sur lequel elle est connectée / !\

Pour la version stream, les résultats de la reconnaissance sont stockés en temps réel dans un fichier json.



Fin du guide …